CVEX v3: Executable CVEs for Verifiable Vulnerability Reproduction

Technical Report

Giovanni Vigna
University of California, Santa Barbara
Santa Barbara, CA, USA
vigna@ucsb.edu

Abstract

A CVE entry tells a defender that a vulnerability exists, but not how to make it happen. Reproducing a vulnerability, that is, rebuilding the vulnerable software, configuring it, and running an exploit that provably succeeds, remains a slow, expert-driven process, and the result is rarely packaged in a form that others can rerun. This is a problem for anyone who needs ground truth about attacks: the developers of detection models, the analysts who assess whether a patch actually fixes a flaw, and the researchers who build and evaluate automated security tools. We present CVEX v3 (*executable CVE*), a standard format for describing the reproduction of a vulnerability as a self-contained, containerized record. A CVEX record bundles the vulnerable target, a benign workload, an exploit, a recorder, and a machine-checkable success criterion. Borrowing from Capture-the-Flag competitions and from the verifiers produced by the CVE-Genie reproduction framework, success is defined by the capture of a secret flag that only a working exploit can reach. Every record is run twice, once with benign traffic and once with the attack, and both runs are captured as packet traces and system-call traces, which yields a labeled, paired dataset. We describe the format and its runner, and release a corpus of 24 records, 10 of which are fully functional and verified, with reference datasets.

Keywords

vulnerability reproduction, CVE, exploits, datasets, containers, intrusion detection, benchmarks

1 Introduction

1.1 The Problem of Vulnerability Reproduction

The Common Vulnerabilities and Exposures (CVE) program assigns an identifier and a short description to every publicly disclosed vulnerability, and the number of entries grows by tens of thousands every year [15, 18]. A CVE entry is a pointer, not an experiment: it names the affected product and versions, and it may link to an advisory or a patch, but it does not tell the reader how to bring the vulnerable software up, how to trigger the flaw, or how to tell whether the attempt worked. Turning an entry into a working reproduction is left to whoever needs it.

Many people need it. Reproductions are the ground truth on which much of security research and practice rests:

- **Detection models.** Network and host intrusion detection systems, signature generators, and machine-learning anomaly detectors must be trained and evaluated on traffic and system activity that contains real, successful attacks, alongside comparable benign activity. Long-lived public datasets are either synthetic or stale, and evaluating detectors on them has been repeatedly criticized [2, 13, 22]. A reproduction that can be rerun at will produces fresh, labeled captures for exactly the vulnerability of interest.
- **Patch analysis.** Security patches are often incomplete: large-scale studies show that a notable fraction of fixes must be revised before a vulnerability is really closed [12]. The two Apache `httpd` path-traversal vulnerabilities in our corpus illustrate this: the fix for CVE-2021-41773 shipped in version 2.4.50 was itself bypassable, which led to CVE-2021-42013. An executable reproduction turns "does the patch work?" into a test that can be run against the patched build, and against every later version.
- **Automated security tools.** Fuzzers, program analyzers, exploit generators, and, more recently, LLM-based agents are all evaluated on vulnerabilities whose presence and exploitability must be established beforehand [9, 25, 28]. Without reproducible environments, these evaluations are hard to compare and hard to repeat.
- **Risk assessment and education.** Whether a vulnerability is exploitable in a given configuration, and what an exploitation looks like on the wire and on the host, are questions that defenders answer most reliably by running the attack. Reproducing attacks is also one of the most effective ways to teach how they work.

Reproduction, however, is expensive. In the most extensive study of the subject, a team of analysts spent 3,600 hours trying to reproduce 368 crowd-reported memory-corruption vulnerabilities, and found that reports routinely omit the information needed to succeed [17].

1.2 The Challenge: No Standard Reproduction Environment

The difficulty is not only in crafting an exploit. Most of the effort goes into the environment: obtaining the exact vulnerable version, building it with a compatible toolchain, installing its dependencies, configuring it into an exploitable state, and seeding it with the data an attack needs. Every reproduction effort solves these problems in its own way, and the solution is rarely captured in a reusable form. Exploit databases publish exploits but not the environment they target [20]. Collections of containerized vulnerable applications, such as Vulhub [24], provide environments but no uniform notion

of what a successful exploitation is, and no instrumentation. Benchmarks built for fuzzers or AI agents define environments tailored to their own harnesses [11, 14, 25, 28]. As a result, there is no standard way to *describe* the environment in which a vulnerability is reproduced, to *verify* that the reproduction succeeded, or to *record* what happened during the attack. Results produced by one group cannot easily be rerun by another, and reproductions are lost when the original setup disappears.

1.3 Our Solution: CVEX, the Executable CVE

We propose CVEX, short for *executable CVE*: a standard format that augments a CVE entry with everything needed to reproduce it. A CVEX record is a small, self-contained directory that describes (i) the vulnerable *target*, built in a container; (ii) a *benign* workload that uses the same service legitimately; (iii) an *attack* that exploits the vulnerability; (iv) a *recorder* that captures network packets and system calls; and (v) a machine-checkable *success criterion*. Running a record produces a verdict and a dataset.

The central design choice of the third version of the format, CVEX v3, is how success is defined. Following Capture-the-Flag (CTF) competitions, and the verifiers produced by the CVE-Genie automated reproduction framework [23], a record plants a secret *flag* in the target, at a location that only a successful exploitation can reach: a root-owned file for a privilege escalation, a file outside the web root for a path traversal, a database row for an SQL injection. The attack succeeds if and only if it returns the flag. This makes verification objective and uniform across vulnerability classes, and it guarantees that the attack half of every dataset contains an exploitation that actually worked.

1.4 Contributions

This report makes two contributions.

(1) **The CVEX v3 format.** We define a uniform, Docker-based record structure with a declarative manifest, three deployment topologies, and three success criteria (planted flag, CTF-style verifier, and crash witness). We define a run protocol that exercises every record in two isolated phases, benign and attack, each against a pristine target, and records both as packet captures and system-call traces. We provide a reference runner that builds a record, runs it, verifies it, and writes a content-addressed dataset (Section 3).

(2) **A corpus of executable CVEs.** We release 24 CVEX v3 records. 10 of them are fully functional: they have been built and run by the reference runner, their exploitation has been verified, and they ship with reference benign/attack captures. These include 10 records migrated by hand from the previous generation of the format and 0 records converted from CVE-Genie reproductions, covering 0 CWE classes across 0 open-source projects. The remaining records are scaffolds that are being functionalized (Section 3.7).

The rest of this report is organized as follows. Section 2 reviews CVEs, CWEs, existing approaches to reproduction, and the earlier versions of CVEX. Section 3 describes the CVEX v3 format, its runner, and the corpus. Section 4 walks through two records in detail. Section 5 discusses extending the format to virtual machines, Section 6 surveys related work, and Section 7 concludes.

2 Background

2.1 CVE and CWE

The CVE program, operated by MITRE on behalf of the U.S. government, assigns a unique identifier of the form `CVE-YYYY-NNNNN` to each publicly disclosed vulnerability, together with a short description and a list of references [15]. The National Vulnerability Database (NVD) enriches CVE entries with severity scores, affected-product configurations, and a weakness classification [18]. The classification uses the Common Weakness Enumeration (CWE), a hierarchical catalog of software and hardware weakness types, such as CWE-22 (path traversal), CWE-79 (cross-site scripting), or CWE-269 (improper privilege management) [16]. CVE and CWE answer the questions *what* is vulnerable and *what kind* of flaw it is; they deliberately do not describe *how* to exploit it. A typical entry consists of a few sentences of prose and links to an advisory, a patch commit, and occasionally a proof of concept (PoC).

2.2 Approaches to Vulnerability Reproduction

Manual reproduction. The traditional approach is manual. An analyst reads the advisory and the patch, locates and builds the vulnerable version, configures it, and writes or adapts an exploit, iterating until the exploit works. Mu et al. [17] quantified the cost of this process: a group of security analysts spent 3,600 hours attempting to reproduce 368 vulnerabilities, and the information in individual reports was often insufficient for success. Their survey of 43 practitioners found that reproduction relies heavily on manual debugging and guesswork. Exploit databases [20] and environment collections such as Vulhub [24] reduce part of this effort, but each covers only half of the problem: one provides exploits without environments, the other environments without a uniform notion of success or instrumentation.

Automated reproduction: CVE-Genie. CVE-Genie [23], to appear at ACM CCS 2026, automates the whole process with a multi-agent system built on large language models. Starting from a CVE identifier, CVE-Genie collects the associated resources (source code at the vulnerable version, advisory, CWE, patch, and PoC when available), rebuilds the vulnerable project, develops an exploit, and produces a *verifier*: a script that runs the exploit and checks its effect. The system is organized in four stages, each made of developer agents paired with critic agents: a Processor, a Builder, an Exploiter, and a CTF Verifier. The verifier is designed after Capture-the-Flag challenges: it prints a fixed flag string only if it observes that the exploit had its intended effect, so that reproduction success can be checked mechanically. Evaluated on 841 CVEs published between June 2024 and May 2025, CVE-Genie reproduced 428 of them (about 51%) across 267 projects, 141 CWEs, and 22 programming languages, at an average cost of $2.77 per CVE. The authors release the reproduced CVEs together with the full logs of the agents' interactions.

CVE-Genie shows that reproduction can be automated at scale, but its outputs are tied to the environment in which the agents worked: a long-lived machine on which the agents installed packages, compiled code, wrote helper files, and started services interactively. The reproduced artifacts (the source snapshot, the exploit, and the verifier) do not capture this state, so they cannot be rerun elsewhere without reconstructing it. Packaging reproductions so

that they can be rerun anywhere is precisely the problem that CVEX addresses, and CVE-Genie's flag-based verifier is one of the main inspirations for the success criterion of CVEX v3.

2.3 Previous Versions of CVEX

CVEX v3 is the fourth iteration of an effort that began at UC Santa Barbara in 2021. Table 1 summarizes the earlier versions.

Version 1 (2021–2022). The first version defined a CVEX entry as a Docker Compose project with three services: a *target* running the vulnerable software, an *exploiter* running the attack, and an optional *collector*, a `tcpdump` sidecar sharing the target's network namespace. A JSON metadata file, validated by a schema, listed the CVEs, the authors, and an `exploit_verification` clause stating how to decide success: a file created or read in the target, a leaked address, or a kernel log message. About fifty entries were produced, and a community scoreboard rewarded contributors with points for CVEs that others had upvoted. Images were served from a project registry, which has since become unavailable, so most of these entries can no longer be run as published.

Version 2 (2023–2024). A team of undergraduate researchers revisited the format with a simpler server/client/listener Compose pattern, published prebuilt images on the GitHub container registry, and catalogued the entries on a website. The version dropped the metadata file and the formal success check: success was established by inspecting the captured traffic.

The VM-based framework (2024–2025). The following version replaced containers with full virtual machines managed by Vagrant and provisioned by Ansible, in order to support OS-level and Windows vulnerabilities. Reusable *blueprints* describe a network of VMs (e.g., an Ubuntu attacker and an Ubuntu target), and per-CVE *records* add the playbooks and the attack commands. A router VM captures all traffic, decrypts TLS with a transparent `mitmproxy`, and the framework traces system calls with `strace` on Linux and API calls with Process Monitor on Windows. Twenty-four records were contributed through pull requests. The framework is powerful, but heavy: every record requires multi-gigabyte VM images and a hypervisor, and, like its predecessors, it has no formal definition of success. Whether an exploit worked is left to the person inspecting the traces.

Lessons. Three lessons from these versions shaped CVEX v3. First, an entry must be rebuildable from what it contains: entries that depended on a registry or on prebuilt images disappeared with them. Second, success must be defined by the record itself and checked by a machine; otherwise a dataset may contain failed attacks labeled as attacks. Third, the format must be light enough to run hundreds of records on a single host, which favors containers over VMs for the large class of vulnerabilities that do not need kernel-level control.

3 CVEX v3

CVEX v3 has three defining properties. First, it is *Docker-only*: a record is a Docker Compose project, with no virtual machines, hypervisor, or per-host provisioning. Second, every record is exercised twice, once with legitimate activity and once with the attack, and both runs are captured as network packets and system calls. Third, success is *flag-based*: the record plants a secret that only a successful exploitation can reach, and the attack succeeds if and only if it retrieves it. The format descends from the previous generations described in Section 2: it keeps the capture sidecar and the explicit exploit verification of the first generation, the server/client container pattern of the second, and the `cvex.yml` manifest and process tracing of the VM-based framework. The flag and the benign/attack pairing are new.

3.1 Record Structure

Every record has the same shape, so that tools and researchers can treat all records uniformly. A record is a directory named after its CVE identifier:

```
CVE-YYYY-NNNNN/
  cvex.yml            # manifest (Sec. 3.3)
  compose.yml         # derived Docker Compose project
  target/
    Dockerfile        # builds the vulnerable software
    vendor/           # pinned sources and binaries
    plant-flag.sh     # installs the flag (Sec. 3.4)
  benign/
    Dockerfile
    interact.sh       # legitimate use of the service
  attacker/
    Dockerfile
    exploit.sh        # the attack; must emit the flag
  recorder/           # shared capture image
  _source/            # provenance: original artifacts
  dataset/            # committed reference run (Sec. 3.7)
```

A record is composed of four container roles, each with a single responsibility (Table 2). A record may add a second target container, typically a database, when the service needs one; everything else stays fixed. The `_source` directory preserves the artifacts from which the record was derived (for example, the previous-generation record or the CVE-Genie reproduction, together with the CVE metadata), so that the provenance of every record is explicit.

The roles are arranged in one of three *topologies*. In the `target-attacker` topology, the benign and attacker clients are separate containers that reach the target over a private network; in `target+db-attacker`, the target is backed by a database container. In the `single` topology, used for local vulnerabilities and for libraries and command-line tools that have no network service, the benign workload and the attack run inside the target container itself, as `target/benign.sh` and `target/attack.sh`.

3.2 Design Rationale

Two choices shape the rest of the format. The first is that a single, uniform notion of success is more valuable than a per-record test. A record whose exploit "ran" tells a user very little: the exploit may have failed silently, or the environment may not have been vulnerable at all. By requiring every attack to produce a value that only a successful exploitation can obtain, the format guarantees that the attack half of every dataset is a genuine, successful exploitation, and makes verification a simple string comparison that is identical across vulnerability classes.

The second choice is to separate the benign and attack phases rather than mixing them. Running the two phases against identical, pristine targets makes the two captures directly comparable, so that the difference between them is the attack itself, and gives clean

Table 1: Evolution of the CVEX format.

	v1 (2021–22)	v2 (2023–24)	VM framework (2024–25)	v3 (2026)
Isolation	Docker Compose	Docker Compose	Vagrant VMs + Ansible	Docker (+ optional services)
Roles	target, exploiter, collector	server, client, listener	target VM(s), attacker VM, router VM	target, benign, attacker, recorder
Descriptor	`docker-compose.yml` + `cvex.json`	`docker-compose.yml`	`cvex.yml` + playbooks	`cvex.yml` + Dockerfiles
Success check	file created/read, leak, `dmesg`	manual inspection	manual inspection	planted flag, verifier, crash witness
Observability	pcap	pcap	pcap, TLS (mitmproxy), strace, Procmon	pcap + strace, benign *and* attack
Platforms	Linux	Linux	Linux and Windows	Linux
Entries	52	6	24	24 (10 functional)

Table 2: Container roles in a CVEX v3 record.

Role	Responsibility
`target`	runs the vulnerable software and holds the planted flag
`benign`	a client that uses the service as a legitimate user would
`attacker`	a client that exploits the service and retrieves the flag
`recorder`	captures packets and system calls during each phase

Listing 1: A `cvex.yml` manifest (demonstration record).

```
cvex_version: 3
cve: CVE-0000-00001
topology: target-attacker # | single | target+db-attacker
network:
  subnet: 10.13.0.0/24    # fixed addresses in the pcap
flag:
  value: "CVEX{00000000000000000000000000000demo}"
  plant: file:/opt/secret/flag
success: flag             # flag | verifier | crash
capture:
  flag_channel: network   # stdout | network | verifier
  tls_intercept: true
  tls_ports: [443]
services:
  target:
    image: cvex/cve-0000-00001-target@sha256:<digest>
    ip: 10.13.0.10
    trace: nginx          # process(es) to trace (regex)
    ready: "tcp:443"      # readiness condition
phases:
  benign:
    client: benign        # runs benign/interact.sh
    image: cvex/cve-0000-00001-benign@sha256:<digest>
    ip: 10.13.0.30
  attack:
    client: attacker      # runs attacker/exploit.sh
    image: cvex/cve-0000-00001-attacker@sha256:<digest>
    ip: 10.13.0.20
limits:
  wall_clock_seconds: 600
```

labels. A *mixed* phase, in which benign activity runs concurrently with the attack, can be added later without changing the schema.

3.3 The Manifest

The `cvex.yml` manifest declares everything the runner needs: the topology, the network layout, the flag and the success criterion, the container images, the processes to trace, and the two phases. Listing 1 shows a manifest for a demonstration record that serves a benign `nginx` instance.

The `ready` condition tells the runner when the target is ready for a phase to start: either a TCP port that accepts connections or a file that the target creates when its initialization completes. The `trace` field is a regular expression that selects the processes whose system calls are recorded. Records converted from CVE-Genie additionally carry an `origin` field and the upstream `source` and `sw_version` of the vulnerable software.

The format imposes a small set of rules, checked at review time: every image is pinned by an `@sha256` digest; either `flag.value` and `flag.plant` are present, or the record declares a different success criterion; both phases are present; and addresses are static, so that captures are reproducible.

3.4 Success Criteria

The flag. A flag is a short token of the form `CVEX{`*32 hex digits*`}`, planted inside the target at a location, or behind a protection, that only the vulnerability can bypass. The secrecy of the flag comes from *access control*, not from its being unknown: a legitimate user of the service cannot reach it, the benign client never sees it, and only a successful exploitation returns it. Because the flag is gated by access rather than by randomness, it can be a fixed value committed with the record, which keeps the reference dataset byte-reproducible. A runner may optionally generate a fresh flag for each run, to guard against an attack that simply hard-codes the expected value; the gating is what proves the exploitation either way.

The location of the flag follows the class of the vulnerability, using a small set of standard placements (Table 3). The `plant` field of the manifest names the location and, where relevant, the file mode, and `plant-flag.sh` installs the flag at container start.

Capture channels. The flag returned by the attack is detected on the channel that the record declares. With the `stdout` channel, the attack prints the flag, and the runner extracts it from the attack's output; this is the universal fallback, and the natural choice for local records. With the `network` channel, the runner searches for the flag in the attack packet capture, which also proves that the flag crossed the wire.

The verifier. Records converted from CVE-Genie use the `verifier` criterion. CVE-Genie produces, for each reproduced CVE, a verifier program in the style of a CTF checker: it sets up the vulnerable condition, runs the exploit, checks for the effect of the vulnerability, and prints a fixed flag if and only if the check passes. A CVEX v3 record that declares `success: verifier` runs this program inside the target during the attack phase, and the runner accepts the reproduction if the verifier exits successfully and prints the flag. The

Table 3: Standard flag placements by vulnerability class.

Class	Flag location
Command execution / injection	`/flag`, readable only by running a command on the target
Path traversal / file read	a file outside the served directory
File upload leading to execution	`/flag`, readable once uploaded content is executed
SQL injection	a table row that the application never exposes
Privilege escalation	`/root/flag`, owned by root, mode 0600
Local code execution / deserialization	`/flag` or an environment value not exposed by the normal interface
Memory corruption (crash only)	none; a crash witness is used instead

verifier criterion extends the flag idea to vulnerabilities whose effect is not the disclosure of a secret, for example a parser that splits one request into two.

The crash witness. Some vulnerabilities, such as memory-corruption bugs that are reproduced only up to a crash, have no secret to read. For these, the record declares `success: crash`, and the flag is replaced by a *crash witness*: the recorder confirms that a traced process terminated because of a signal, or produced a core dump, during the attack phase and not during the benign phase. This is the only deviation from the flag rule, and it is declared explicitly.

In the current corpus, 9 functional records use a planted flag (5 on the `stdout` channel and 4 on the `network` channel), 0 use a CVE-Genie verifier, and 1 uses a crash witness.

3.5 Run Protocol

A record is run by the reference runner in two isolated phases, each starting from a *pristine* target: the target containers are created fresh for each phase, so that neither run can contaminate the other.

(1) **Build.** The runner builds (or pulls) the images.
(2) **Benign phase.** The runner starts the target (and database, if any) and the recorder, plants the flag, waits for the `ready` condition, starts the capture, runs the benign workload, stops the capture, and tears the containers down.
(3) **Attack phase.** The runner starts a fresh target and recorder, plants the flag, waits for readiness, starts the capture, runs the attack, collects the flag it emits, stops the capture, and tears the containers down.
(4) **Verify.** The runner compares the observed flag with the planted one (or evaluates the verifier or the crash witness) and writes the dataset and the verdict.

The benign workload performs legitimate interactions with the same service and, where possible, with the same endpoints that the attack uses: a normal login, a normal search, a normal file request, a normal invocation of a tool. The two captures are therefore directly comparable: same images, same addresses, same target state at the start of the phase. Each phase is bounded by the wall-clock limit declared in the manifest.

A second command reruns a record and compares the fresh dataset with the committed one, checking that the flags match and comparing the traffic after normalizing timestamps and ephemeral ports. Once a record has been verified in this way, it is locked: a lock file records the hash of the verified record, so that later modifications are detected.

Listing 2: The `verdict.json` of a functional record.

```
{
  "cve": "CVE-2023-22809",
  "success": true,
  "criterion": "flag",
  "channel": "stdout",
  "flag_planted":  "CVEX{9b8bb059600aa99e8a28eca00bb4ddef}",
  "flag_observed": "CVEX{9b8bb059600aa99e8a28eca00bb4ddef}",
  "benign": { "pcap": true, "syscalls": true },
  "attack": { "pcap": true, "syscalls": true }
}
```

3.6 The Recorder

The recorder is a single shared image. In *sidecar* mode, it shares the network namespace and the process namespace of the target container. It runs `tcpdump` on the target's interfaces, including the loopback interface, so that it also captures traffic between processes inside the target, and it attaches `strace` to the processes selected by the `trace` expression, following their children. In *gateway* mode, intended for multi-host records that use TLS, the recorder acts as the default gateway of the record's network, captures traffic on the bridge, and runs a transparent `mitmproxy` whose certificate authority is installed in the client and target images at build time, so that encrypted traffic can be decrypted reproducibly.

3.7 Dataset Layout

The output of a run is a self-contained dataset directory:

```
dataset/
  benign/
    benign.pcap          # legitimate traffic
    benign.syscalls.log  # system calls under normal use
  attack/
    attack.pcap          # attack traffic
    attack.syscalls.log  # system calls during the attack
  flag.observed          # the flag the attack retrieved
  verdict.json           # criterion, channel, planted vs.
                         # observed flag, success
  MANIFEST.sha256        # content hashes of all files
```

For local records the packet captures may be empty or contain only loopback traffic, and the system-call traces carry most of the signal. The verdict records the success criterion, the capture channel, the planted and the observed flag, the outcome, and which captures are present. Listing 2 shows the verdict of the privilege-escalation record discussed in Section 4. The dataset is committed with the record and addressed by the hashes in `MANIFEST.sha256`, so that the reference captures survive even if the images can no longer be rebuilt.

3.8 Reproducibility

The format pursues reproducibility at four levels. Builds are meant to be offline and pinned: base images are referenced by digest, and the vulnerable software is vendored into the record, so that the application layer can be built without network access. The runtime is pinned: a static subnet and static addresses, explicit readiness conditions, and a wall-clock limit. The flag is fixed and

Table 4: The two example records.

	CVE-2023-22809	CVE-2024-52304
Software	sudo 1.9.12p1	aiohttp 3.10.10
CWE	CWE-269	CWE-444
Class	local privilege escalation	HTTP request smuggling
Origin	previous CVEX generation	CVE-Genie
Topology	single	single
Success criterion	planted flag	CVE-Genie verifier
Flag location	/root/flag (mode 0600)	—
Traced process	sudo	python
Attack capture	2,164 syscalls	515 KB syscalls, 4.5 KB pcap

committed, so that reference captures are stable while still proving the exploitation. Finally, the reference dataset is committed and content-addressed.

Not all of these properties hold yet for every record. Images are not yet pinned by digest, and records converted from CVE-Genie still download the vulnerable software at build time rather than vendoring it. Packet timing is not reproducible in any case: payloads and the order of exchanges are, and timing is normalized away when datasets are compared.

3.9 Limitations

Because containers share the host kernel, the format cannot reproduce vulnerabilities in the kernel itself, or in operating systems other than Linux; Section 5 discusses an extension to virtual machines that shares the same manifest and dataset format. TLS interception assumes that the client and target trust the recorder's certificate authority, which is acceptable for research reproductions and is noted in each dataset. Finally, the realism of the benign phase is defined by the record's author: the value of the benign baseline depends on how representative the benign workload is, and reviewers should check that it exercises the same interface as the attack.

4 Examples

We describe two functional records from the corpus. Both concern well-known, publicly disclosed and long-patched vulnerabilities; the point of the records is not the vulnerabilities themselves, but the reusable, verified datasets they produce. The first, a local privilege escalation in sudo, was migrated by hand from the previous generation of CVEX and is verified by a planted flag. The second, an HTTP request-smuggling flaw in the aiohttp Python framework, was converted from a CVE-Genie reproduction and is verified by CVE-Genie's CTF-style verifier. Table 4 summarizes them.

4.1 Local: sudoedit (CVE-2023-22809)

The vulnerability. sudoedit (sudo -e) lets an administrator allow a user to edit specific files with elevated privileges, without granting a root shell. According to the public advisory, versions of sudo up to 1.9.12p1 mishandle the editor selected through the user's environment, so that a user who is permitted to edit one file can edit other files as well [15]. Since the user controls which files are opened with elevated privileges, the flaw amounts to a full privilege escalation. It was fixed in sudo 1.9.12p2.

The target. The target image builds the vulnerable sudo 1.9.12p1 from a vendored source tarball, so that the record does not depend on a distribution package that may change or disappear. It creates an unprivileged user and installs a sudoers policy that grants this user a single right: to edit /etc/motd with sudoedit. At container start, plant-flag.sh writes the record's flag to /root/flag, owned by root with mode 0600. The policy is the whole point of the record: under a correct sudo, nothing the user is allowed to do can reveal the flag.

Benign and attack phases. The benign phase exercises exactly the permission the policy grants: the user edits /etc/motd through sudoedit, with an editor that leaves the file unchanged. The attack phase starts from a fresh container and runs, as the same unprivileged user, the publicly documented exploitation of the flaw; it succeeds if it prints the flag. Both phases run the same binary under the same policy, so the two traces differ only in what the exploitation adds.

Verification and dataset. The record declares success: flag with the stdout channel: the runner compares the flag printed by the attack with the planted one, and in the reference run they match. Because the flag is readable only by root, a match proves that the escalation genuinely happened, and not merely that an exploit ran. The record is local, so its dataset is centered on system calls: the recorder traces sudo and its children, producing 1,699 lines in the benign phase and 2,164 in the attack phase. The difference is visible directly in the traces: the attack trace contains the access to /root/flag and additional process executions, while the benign trace never touches the flag. This pairing (same program, same policy, one benign and one malicious use) is the kind of labeled data that host-based detectors need.

4.2 Network-Facing: aiohttp (CVE-2024-52304)

The vulnerability. aiohttp is a widely used asynchronous HTTP client/server framework for Python. According to its advisory, before version 3.10.11 the pure-Python HTTP parser incorrectly handled newlines inside chunk extensions of chunked request bodies [15]. When the C extensions are not in use, a front-end proxy and an aiohttp back end can disagree on where a request ends, which enables HTTP request smuggling (CWE-444) and can let an attacker bypass protections enforced by the proxy.

From a CVE-Genie reproduction to a record. CVE-Genie reproduced this CVE automatically, producing an exploit and a verifier. The verifier forces the pure-Python parser, starts an aiohttp server, runs the exploit against it, and checks whether the server received two distinct requests although the client sent only one; if so, it prints CVE-Genie's flag. The conversion to CVEX v3 packages these artifacts into a record: the target image is python:3.11-slim with aiohttp 3.10.10 (the last release before the fix) installed from PyPI, and the exploit and verifier are copied into it. The record's _source directory keeps CVE-Genie's original artifacts and the CVE metadata (advisory, CWE, patch commit), so that the record's provenance is preserved.

Verification and dataset. The record declares success: verifier: the attack phase runs CVE-Genie's verifier inside the target, and

the runner accepts the reproduction if the verifier succeeds; in the reference run it prints the flag `3xploit66full`. Because the verifier starts its own server and talks to it over the loopback interface, the recorder, which shares the target's network namespace, captures the smuggling exchange itself (a 4.5 KB packet capture) together with 515 KB of system-call trace of the Python processes. The record is a network-facing vulnerability reproduced within a single container: no separate attacker container is needed, yet the dataset still contains the malicious HTTP traffic.

Limitations. This record also shows where the corpus still needs work. Its benign phase only imports the library, a placeholder rather than a realistic workload; for most records converted from CVE-Genie, the benign workloads have yet to be written so that they exercise the same interface as the attack (here, normal chunked requests to the same server). Records converted from CVE-Genie also download their source at build time rather than vendoring it. Both are on our list of improvements to the corpus.

5 Future Work: Virtual Machines

Containers share the host's kernel. This is what makes CVEX v3 records light enough to run by the hundred on one machine, but it also bounds the vulnerabilities the format can express. A record cannot install a different kernel, load kernel modules, change kernel configuration, or run a non-Linux operating system, and a privileged operation inside a container affects the host as well. We plan to extend the format with a *VM target*: a manifest option that replaces the target container with a virtual machine image, while keeping everything else in the record unchanged (the roles, the success criteria, the two-phase run protocol, and the dataset layout). The VM-based generation of CVEX (Section 2) already demonstrated how to provision and instrument such targets; the extension would bring that capability into the uniform CVEX v3 record.

Several classes of vulnerabilities need this extension:

- **Kernel vulnerabilities.** Linux kernel flaws such as CVE-2016-5195 ("Dirty COW"), CVE-2022-0847 ("Dirty Pipe"), and CVE-2024-1086 (a use-after-free in `nf_tables`) depend on a specific vulnerable kernel version and configuration, which a container cannot provide.
- **Vulnerabilities in non-Linux systems.** Windows vulnerabilities such as CVE-2017-0144 (the SMBv1 flaw known as EternalBlue), CVE-2019-0708 ("BlueKeep", in the Remote Desktop service), and CVE-2021-34527 ("PrintNightmare", in the Print Spooler) require a Windows guest.
- **Container and isolation boundaries.** Vulnerabilities in the container runtime itself, such as CVE-2019-5736 in `runc`, cannot be reproduced safely inside the runtime they affect.
- **Environment-dependent behavior.** Some user-space vulnerabilities depend on how the C library and the kernel interact. While building the corpus we were unable to reproduce CVE-2019-18276, a `bash` privilege-management flaw, because it requires a C library and kernel combination that preserves the saved user ID when `bash` drops privileges, which our container environment does not provide.
- **Records that need host privileges.** A few records in the corpus already require additional container capabilities (for example, permission to mount file systems or to configure traffic control) to reproduce their vulnerability. Granting these weakens the isolation between the record and the host; a VM target would restore it.

A VM target also enables richer observation. Besides packets and system calls, a VM-based recorder can capture kernel events, Windows API calls, and whole-system snapshots taken before and after the attack. The success criteria of CVEX v3 carry over unchanged: a flag readable only by root in the guest, or a crash witness for a kernel panic, decides success just as it does for containers.

6 Related Work

Studies of reproducibility. Mu et al. [17] conducted the first large empirical study of vulnerability reproducibility, measuring how often crowd-reported memory-corruption vulnerabilities can be reproduced from public reports and which missing information most often prevents it. Their findings motivate formats, like CVEX, that capture the full environment of a reproduction rather than a prose description of it.

Reproducible vulnerability datasets. ARVO [14] rebuilds thousands of memory-safety vulnerabilities found by OSS-Fuzz in C/C++ projects, providing for each a build environment, a triggering input, and the fixing patch. Magma [9] forward-ports real bugs into recent library versions, with ground-truth canaries, to benchmark fuzzers. Vulhub [24] is a community collection of Docker Compose environments for vulnerable applications. In software engineering, Defects4J [10] established the value of a curated, executable database of real faults for controlled experiments. CVEX shares the goal of executable ground truth, but differs in three respects: it is not limited to a single bug class or ecosystem, it defines success uniformly through a planted flag, and it records paired benign and attack activity at both the network and the system-call level.

Benchmarks for automated exploitation. The rise of LLM-based agents has produced several benchmarks built on real vulnerabilities. CVE-Bench [28] packages 40 critical web application CVEs in a sandbox that scores agents' exploitation attempts. Cybench [26] collects 40 professional Capture-the-Flag tasks. CyberGym [25] asks agents to generate PoCs for 1,507 vulnerabilities from 188 projects, given a description and the codebase, and SEC-bench [11] automatically constructs PoC-generation and patching tasks from real vulnerabilities. Earlier, the DARPA Cyber Grand Challenge [3] and the AI Cyber Challenge [4] evaluated autonomous systems on purpose-built or injected vulnerabilities. These benchmarks define environments and success checks for their own harnesses; CVEX aims to be a harness-independent format, and its records can serve as tasks for such benchmarks.

Automated reproduction. CVE-Genie [23] automates the end-to-end reproduction of CVEs, from collecting resources to building the project, exploiting it, and verifying the result with a CTF-style verifier. CVEX is complementary: it provides the portable, rerunnable packaging that turns an automated reproduction into a durable artifact, and 0 records of our corpus were obtained by converting CVE-Genie's reproductions.

Vulnerability code datasets. A large body of work collects vulnerable and fixed code for training and evaluating vulnerability

detectors, including the Juliet test suite in NIST's SARD [19], Devign [27], BigVul [6], CVEfixes [1], and PrimeVul [5]. These datasets are static: they label code, but do not provide a way to run the vulnerable program or to demonstrate exploitation. CVEX records can complement them with executable evidence.

Intrusion detection datasets. Labeled traffic and system-call datasets are the traditional basis for evaluating intrusion detection, from the DARPA 1999 evaluation [13] to ADFA-LD [2], CICIDS2017 [21], and LID-DS [8]. Sommer and Paxson [22] discuss the pitfalls of evaluating machine-learning detectors on such data. CVEX produces per-vulnerability, paired benign/attack captures on demand, where the attack half is guaranteed by the success criterion to contain a working exploitation.

Kernel and OS-level reproduction. Reproducing kernel vulnerabilities requires control over the kernel itself. syzbot [7] continuously reproduces kernel bugs found by fuzzing in virtual machines, and SyzBridge [29] adapts upstream PoCs to downstream distribution kernels to assess their exploitability. These systems motivate the VM extension of CVEX discussed in Section 5.

7 Conclusion

A CVE entry records that a vulnerability exists; it does not let anyone observe it. We presented CVEX v3, a standard format for executable CVEs that packages the vulnerable target, a benign workload, an attack, a recorder, and a machine-checkable success criterion into a self-contained, container-based record. Inspired by Capture-the-Flag competitions and by the verifiers of CVE-Genie, CVEX v3 defines success as the capture of a secret that only a working exploit can reach, and it runs every record twice, so that each run yields a labeled pair of benign and attack captures at the network and system-call level.

We release a corpus of 24 records, 10 of which are fully functional and ship with verified reference datasets. We hope that CVEX will make reproductions durable and shareable artifacts, and that the corpus will be useful to researchers who build detection models, assess patches, and evaluate automated security tools. Functionalizing the remaining records, and extending the format to virtual machines for vulnerabilities that require OS-level control, are the next steps.

References

[1] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software. In *Proceedings of the International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*.

[2] Gideon Creech and Jiankun Hu. 2013. Generation of a New IDS Test Dataset: Time to Retire the KDD Collection. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC)*.

[3] DARPA. 2016. Cyber Grand Challenge (CGC). https://www.darpa.mil/research/programs/cyber-grand-challenge.

[4] DARPA. 2025. AI Cyber Challenge (AIxCC). https://aicyberchallenge.com.

[5] Yangruibo Ding et al. 2025. Vulnerability Detection with Code Language Models: How Far Are We?. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*.

[6] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*.

[7] Google. [n. d.]. syzbot: Continuous Kernel Fuzzing. https://syzkaller.appspot.com. Accessed 2026-10.

[8] Martin Grimmer et al. 2023. Dataset Report: LID-DS 2021. In *Critical Information Infrastructures Security (CRITIS 2022) (Lecture Notes in Computer Science, Vol. 13723)*. Springer, 63–73.

[9] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A Ground-Truth Fuzzing Benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3 (2020).

[10] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*.

[11] Hwiwon Lee, Ziqi Zhang, Hanxiao Lu, and Lingming Zhang. 2025. SEC-bench: Automated Benchmarking of LLM Agents on Real-World Software Security Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[12] Frank Li and Vern Paxson. 2017. A Large-Scale Empirical Study of Security Patches. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*.

[13] Richard Lippmann, Joshua W. Haines, David J. Fried, Jonathan Korba, and Kumar Das. 2000. The 1999 DARPA Off-Line Intrusion Detection Evaluation. *Computer Networks* 34, 4 (2000).

[14] Xiang Mei et al. 2026. ARVO: Atlas of Reproducible Vulnerabilities for Open-Source Software. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*. arXiv:2408.02153.

[15] MITRE Corporation. [n. d.]. Common Vulnerabilities and Exposures (CVE). https://www.cve.org. Accessed 2026-10.

[16] MITRE Corporation. [n. d.]. Common Weakness Enumeration (CWE). https://cwe.mitre.org. Accessed 2026-10.

[17] Dongliang Mu, Alejandro Cuevas, Limin Yang, Hang Hu, Xinyu Xing, Bing Mao, and Gang Wang. 2018. Understanding the Reproducibility of Crowd-reported Security Vulnerabilities. In *Proceedings of the USENIX Security Symposium*.

[18] National Institute of Standards and Technology. [n. d.]. National Vulnerability Database (NVD). https://nvd.nist.gov. Accessed 2026-10.

[19] National Institute of Standards and Technology. [n. d.]. Software Assurance Reference Dataset (SARD). https://samate.nist.gov/SARD. Accessed 2026-10.

[20] OffSec. [n. d.]. Exploit Database. https://www.exploit-db.com. Accessed 2026-10.

[21] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*.

[22] Robin Sommer and Vern Paxson. 2010. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In *Proceedings of the IEEE Symposium on Security and Privacy*.

[23] Saad Ullah, Praneeth Balasubramanian, Wenbo Guo, Amanda Burnett, Hammond Pearce, Christopher Kruegel, Giovanni Vigna, and Gianluca Stringhini. 2026. From CVE Entries to Verifiable Exploits: An Automated Multi-Agent Framework for Reproducing CVEs. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. To appear. arXiv:2509.01835.

[24] Vulhub contributors. [n. d.]. Vulhub: Pre-Built Vulnerable Environments Based on Docker Compose. https://github.com/vulhub/vulhub. Accessed 2026-10.

[25] Zhun Wang, Tianneng Shi, et al. 2026. CyberGym: Evaluating AI Agents' Real-World Cybersecurity Capabilities at Scale. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[26] Andy K. Zhang et al. 2025. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[27] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[28] Yuxuan Zhu et al. 2025. CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities. In *Proceedings of the International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*.

[29] Xiaochen Zou, Yu Hao, Zheng Zhang, Juefei Pu, Weiteng Chen, and Zhiyun Qian. 2024. SyzBridge: Bridging the Gap in Exploitability Assessment of Linux Kernel Bugs in the Linux Ecosystem. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.